SCOPE: Structured Code Orchestration via Prompt Engineering for Web App Generation

Matteo Vigone[a], Alessandro Francesco Maria Martina[b,a,*], Cataldo Musto[a] and Giovanni Semeraro[a]

[a] University of Bari Aldo Moro, Via E. Orabona, Bari, 70125, Italy
[b] University of Pisa, Largo B. Pontecorvo, Pisa, 56172, Italy

ARTICLE INFO

Keywords:
Large Language Models
Software Engineering
Web Applications

ABSTRACT

Automating the end-to-end development of web applications remains a critical challenge in software engineering. Current autonomous-agentic frameworks often struggle with architectural consistency, vague requirements, and error propagation across multi-phase workflows for software generation. To address these limitations, this paper introduces SCOPE, a structured code orchestration framework that addresses these gaps and generates structurally consistent, specification-aligned web applications. Central to our approach is Extended Self-Planning, a novel methodology that operationalizes the traditional Software Development Life Cycle, spanning requirements analysis, architectural design, and task decomposition before code generation. By integrating In-Context Learning, Chain-of-Thought, and Chain-of-Code prompting within a sequential Division of Labor pipeline, the framework ensures logical continuity across the application's layers. To validate this approach, we introduce Structural and Functional Coherence Evaluation, a multidimensional methodology for assessing the functional and structural integrity of AI-generated software. Empirical validation across 25 full-stack applications generated by SCOPE demonstrates that the framework achieves high execution accuracy and functional coherence.

1. Introduction

Developing modern web applications remains complex, often leading to time and cost overruns despite agile practices (Dikert et al., 2016). This complexity stems not only from implementation effort, but also from the need to design and maintain coherent software architectures across multiple interacting components. Attempts to reduce this complexity by increasing the abstraction level of programming languages have reached structural limits (Terragni et al., 2025). Consequently, research has shifted toward automating code generation directly from natural-language instructions (Shin et al., 2021).

The shift from rule-based (Reddy et al., 2015) systems to Transformer-based LLMs pretrained on large text–code corpora (Vaswani et al., 2017; Radford et al., 2019; Brown et al., 2020; Feng et al., 2020; Svyatkovskiy et al., 2020; Wang et al., 2021), has enabled substantial progress in automated code generation. Modern LLMs can translate natural-language instructions into functional code and deliver measurable productivity gains (Guo et al., 2023; Lu et al., 2022). Despite these advances, their use remains confined to isolated coding tasks. They lack mechanisms to maintain cross-file context and to coordinate multi-step workflows, both of which are essential for full-stack web development. They are also prone to factual inaccuracies and hallucinations (Tonmoy et al., 2024; Kalai et al., 2025), often misinterpreting ambiguous requirements.

To mitigate these limitations, agentic architectures have been developed to structure and coordinate LLM-driven code generation. Notable strategies include self-planning, which decomposes high-level goals into intermediate steps prior to synthesis (Jiang et al., 2024; Gur et al., 2024; Bairi et al., 2024); retrieval-augmented generation, which grounds the process in external knowledge (Phan et al., 2024; Acharya et al., 2025; Gao et al., 2023); and self-improvement loops, which iteratively refine generated code through automated feedback (Madaan et al., 2023; Zhang et al., 2023; Chang et al., 2023; Chen et al., 2024c).

*Corresponding author
m.vigone@studenti.uniba.it (M. Vigone); alessandro.martina@phd.unipi.it (A.F.M. Martina); cataldo.musto@uniba.it (C. Musto); giovanni.semeraro@uniba.it (G. Semeraro)
(M. Vigone); (A.F.M. Martina); (C. Musto); (G. Semeraro)
ORCID(s):

In the recent literature, multi-agent systems (MAS) have also emerged, distributing development tasks across specialized agents. Among the various approaches proposed, the most widely adopted fall into three categories: Self-Negotiation (Islam et al., 2024; Nunez et al., 2024; Hu et al., 2025), Pipeline-Based Division of Labor (DoL) (Rasheed et al., 2024; Huang et al., 2023), and Hierarchical Planning (Zhang et al., 2024).

While both single-agent and multi-agent approaches advance task-level code quality, they share a common limitation. They do not explicitly address architectural reasoning, such as maintaining consistency across components, enforcing design constraints, or preserving system-level coherence (Dong et al., 2025). Individual components may be locally correct yet fail to integrate into a coherent architecture. This limitation is particularly critical in full-stack web development, where database schemas, backend logic, and frontend interfaces must remain mutually aligned (Pezeshkpour et al., 2024; Dong et al., 2025).

These shortcomings expose three open challenges that motivate the present work. First, current approaches typically address narrow tasks (e.g., file-level generation or the creation of small software programs) and struggle to maintain context across complex architectures and multi-step development processes (Yang et al., 2021; Chen et al., 2024a). Second, LLM-based systems are highly sensitive to prompt quality and often fail when faced with ambiguous, high-level, or incomplete user requirements, limiting their use in real-world scenarios (Odeh et al., 2024). Third, there is no widely adopted methodology for assessing the completeness, consistency, and reliability of entire AI-generated web applications, making it difficult to compare systems or validate their effectiveness (Evtikhiev et al., 2023).

We observe that existing pipelines typically collapse the entire development lifecycle into a single generation step or, at best, a plan-then-code paradigm (Jiang et al., 2024). This conflation forces the model to simultaneously resolve requirements ambiguity, make architectural decisions, and produce implementation code tasks that, in traditional software engineering, are deliberately handled in distinct phases. This observation motivates a framework that explicitly separates these concerns, orchestrating the full development lifecycle from requirement analysis through cross-layer code synthesis, while providing a rigorous evaluation protocol for the resulting artifacts. To this end, this work aims to tackle the above-mentioned issues by providing the following contributions:

- **SCOPE**: A structured code orchestration framework that automates full-stack web application development from a single natural language specification. The framework introduces **Extended Self-Planning**, which operationalizes the Software Development Life Cycle (SDLC) (Jain and Suman, 2015) by explicitly separating requirements analysis, architectural design, and task decomposition prior to code generation. It integrates In-Context Learning (ICL), Chain-of-Thought (CoT), and Chain-of-Code (CoC) prompting within a pipeline-based division-of-labor architecture, enforcing architectural constraints and preserving cross-layer consistency.
- **SFCEval**: A systematic, multi-dimensional evaluation methodology designed to assess the functional correctness, completeness, and architectural coherence of entire AI-generated web applications, addressing the lack of standardized evaluation frameworks for system-level generation.
- **Empirical Validation**: A comprehensive empirical study evaluating the proposed approach across multiple LLMs. This includes a comparative analysis, an ablation study isolating the contribution of the requirements refinement stage, and an assessment of the framework's ability to generate structurally consistent and functionally correct applications.

The remainder of this paper is organized as follows: Section 2 reviews related work; Section 3 presents the *SCOPE* framework; Section 4 describes the evaluation methodology; Section 5 discusses the experimental results; and Section 6 concludes the paper with limitations and future research directions.

2. Related Work

2.1. Foundational Techniques in LLM-based Code Generation

The effectiveness of LLMs in software engineering depends not only on model size or pretraining corpus, but also on prompting strategies, reasoning techniques, and self-planning methods that align models with real-world workflows (Dong et al., 2025).

Advanced prompting methods, such as CoT, Structured CoT (SCoT), and CoC (Wei et al., 2022; Li et al., 2025, 2024b), allow LLMs to externalize reasoning steps and integrate algorithmic logic, improving code accuracy and interpretability. CoT decomposes complex tasks into sequential sub-steps, reducing logical errors in multi-step generation. CoC extends this by interleaving natural language reasoning with code-level constructs such as

pseudocode and partial implementations, improving alignment between intended functionality and generated code. Tree of Thoughts (ToT) (Yao et al., 2023) further generalizes CoT by exploring multiple reasoning paths in parallel and selecting the most promising one through self-evaluation. While ToT demonstrates that branching exploration can substantially improve single-task problem solving, it still operates within the boundary of a single generation call. None of these techniques provides a mechanism to coordinate reasoning *across* multiple dependent tasks, such as ensuring that a database schema, the API that queries it, and the frontend that consumes it are mutually consistent.

The broader ICL paradigm (Dong et al., 2024) is a natural extension of these prompting techniques. In practice, ICL serves as the primary mechanism by which pretrained LLMs adapt to new tasks at inference time, leveraging examples within the context window. (Dong et al., 2024; Mosbach et al., 2023; Wang et al., 2023). Its limitation is that the examples are static: they define structural patterns for one task at a time but cannot encode cross-task dependencies.

Building on this ability, recent work has explored mechanisms to explicitly structure the generation process. Self-planning (Jiang et al., 2024) is one such approach, in which the model first generates a high-level plan to guide subsequent code generation. This method aligns well with requirements engineering by leveraging abstraction and decomposition to better match the generated code to architectural coherence. However, the method relies on a single planning step that conflates requirements analysis, architectural design, and task breakdown into one monolithic output. As a result, the model must simultaneously resolve requirement ambiguities and make structural decisions without the benefit of intermediate validation.

Self-refine (Madaan et al., 2023) and self-edit (Zhang et al., 2023) address code quality through iterative feedback loops: the model generates code, evaluates it, and refines it until requirements are met. Reflexion (Shinn et al., 2023) advances this paradigm by maintaining an episodic memory of verbal reflections from previous failed attempts. This improves convergence on individual tasks. However, reflections remain scoped to the current task: the agent reasons about why a function fails, not about whether it is consistent with the broader system. All iterative methods share structural limits in multi-component settings. Each cycle operates on a single file without visibility into dependent components: a backend fix is not propagated to the frontend that consumes it. Additionally, their iterative nature is potentially redundant and can break the continuity of a structured pipeline, making them unsuitable for orchestrating complex, multi-component software.

In summary, existing prompting and reasoning techniques improve generation quality within individual tasks but do not address cross-task coordination. They lack mechanisms to propagate architectural decisions across pipeline stages or to verify consistency between interdependent components.

2.2. Orchestration Frameworks for LLM-based Code Generation

Frameworks using intelligent agents have advanced automated code generation, but common challenges remain in scalability and ambiguity management.

Several frameworks have focused on simulating software development through different multi-agent collaboration models. ChatDev (Qian et al., 2024), for instance, simulates a programming team with five distinct agent roles (e.g., design, coding, testing). Collaboration is implemented through structured dialogues between pairs of agents, each constrained to domain-specific tasks. However, collaboration occurs in isolated pairwise sessions: the dialogue between the designer and the coder is not visible to the subsequent coder-tester interactions. Since no shared architectural artifact constrains all agents, cross-component consistency depends entirely on what each agent retains from prior dialogues rather than on an explicit global specification.

MetaGPT (Hong et al., 2024) advances this paradigm by assigning explicit software engineering roles (product manager, architect, engineer) and enforcing structured outputs through standardized operating procedures (SOPs). The architect produces a design document that downstream agents should follow. However, the framework lacks a verification mechanism to ensure that the generated code actually adheres to the architectural specification. The SOPs structure the development process but do not enforce consistency between design artifacts and implementation. Moreover, meeting the complex and diverse requirements of real-world applications remains challenging for the framework.

A different approach, CodeCoR (Pan et al., 2025), implements a self-reflective pipeline of four specialized agents (Prompt, Test, Coding, and Repair). Its strength lies in a self-reflective mechanism based on multiple generations, which aims to reduce error propagation. Its repair mechanism targets local defects such as syntax errors or failing tests, but does not verify coherence across components. If the Repair agent modifies a backend API signature, this change is not propagated to the frontend modules that consume it. The self-repair loop is therefore local in scope, not architectural.

However, its reliance on iterative pruning may introduce latency and does not explicitly address cross-component integration.

To address scalability, the Self-Organized Agents (SoA) framework (Ishibashi and Nishimura, 2024) introduces a hierarchical model. This self-organizing structure is designed to manage complexity by keeping the workload and short-term memory requirements for any single agent. Although effective for scalability, this approach focuses on task distribution rather than maintaining architectural coherence across generated components. While effective for workload distribution, each sub-agent generates its output independently. There is no sequential context propagation, for instance, from the database schema to backend logic to the frontend interface, so cross-layer alignment is not guaranteed.

Managing ambiguous or underspecified requirements remains a major challenge. Some frameworks, such as (Vijayvargiya et al., 2025), introduce agents that detect ambiguity and query proxy users to fill information gaps, while ChatDev employs "communicative dehallucination" (Qian et al., 2024) to ask clarifying questions before generating a response. These approaches reduce hallucinations but rely on explicit triggers and cannot fully compensate for intrinsic model knowledge gaps. On the other hand, HiLDe (Human-in-the-Loop Decoding) (Gonzalez et al., 2025) integrates the user in code generation by presenting alternatives and explanations, allowing the user to guide regeneration. While effective for intentional coding, this interactivity can increase completion time and risk unintended changes. Moreover, these approaches do not ensure that resolved ambiguities translate into consistent architectural decisions across the system.

In summary, existing frameworks either lack shared architectural artifacts that constrain all generation stages or rely on local repair mechanisms that do not propagate corrections across components. None enforces lifecycle-driven planning that explicitly separates requirements analysis, architectural design, and task decomposition prior to code generation. Crucially, none of the reviewed frameworks produces a complete three-tier web application — comprising a relational database, a backend API, and a frontend client — as an integrated, executable system. As Table 1 shows, existing approaches target either single-component generation or narrowly scoped multi-file outputs, without guaranteeing cross-layer architectural consistency. This difference in output scope has direct implications for evaluation: metrics and benchmarks designed for function-level or file-level assessment cannot capture the system-level properties that full-stack generation demands. These implications are addressed in Section 2.3 and operationalized in the evaluation design (Section 4). Table 1 compares their architectures and functionalities with SCOPE, which orchestrates the full software lifecycle, integrates multiple reasoning stages, and ensures the delivery of consistent, complete, full-stack web applications.

Table 1
Comparison of LLM-based code generation frameworks by architectural features and development scope. *Multi-Agent* denotes multi-agent coordination; *Full-Stack* indicates support for database, backend, and frontend generation; *Lifecycle Planning* refers to explicit planning across development phases; *End-to-End* indicates full pipeline support from requirements to executable code.

Framework	Multi-Agent	Full-Stack	Lifecycle Plan	End-to-End
ChatDev	yes	no	Partial	no
CodeCoR	yes	no	Repair-based	no
SoA	yes	no	Dynamic	no
HiLDe	no	no	no	no
metaGPT	yes	no	Partial (SOP)	no
SCOPE (ours)	no	yes	**Extended self-planning**	yes

2.3. Evaluation Methodologies for LLM-based Code Generation

Recent literature categorizes evaluation metrics into three main types: similarity-based, execution-based, and feedback-based (Chen et al., 2024b). Each addresses a specific aspect of code quality, but none captures the system-level properties required for full-stack application assessment.

Similarity-Based Metrics. Similarity-based methodologies compare the generated code against a reference solution. While traditional metrics from Natural Language Processing, like BLEU (Papineni et al., 2002), ROUGE (Lin, 2004), and METEOR (Banerjee and Lavie, 2005), are used, modern benchmarks have introduced code-specific metrics such

as CodeBLEU (Ren et al., 2020), which offers a more accurate assessment by incorporating syntactic and semantic code features.

However, these metrics suffer from two structural limitations. First, they require a gold-standard reference solution, which does not exist for full-stack applications generated from open-ended natural language prompts. Second, they operate on individual files in isolation: they cannot detect whether a backend endpoint is consistent with the frontend API call that consumes it, because each file is scored independently without cross-file context (Evtikhiev et al., 2023).

Execution-Based Metrics. Execution-based metrics assess the practical functionality of generated software, with Pass@k (Chen et al., 2021) being the most widely adopted. It measures the probability that at least one of k solutions passes all unit tests, forming the basis for benchmarks like HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021). More recent benchmarks extend this paradigm: SWE-bench (Jimenez et al., 2024) evaluates issue resolution in real-world repositories, while DevBench (Li et al., 2024a) assesses LLM performance across multiple development phases including design, coding, and testing.

Beyond correctness, efficiency is evaluated in terms of runtime, memory usage, and code complexity using benchmarks such as EffiBench (Huang et al., 2024) and Mercury (Du et al., 2024). Despite their rigor, these benchmarks share a common structural limitation: they evaluate correctness at the function level or the issue level, not at the system level. Code that passes all unit tests for individual modules can still produce a non-functional application if the interfaces between database, backend, and frontend layers are incompatible. No existing execution-based benchmark tests the integrated behavior of a complete multi-layer application.

Feedback-Based Metrics. Feedback-based evaluation relies on human judgment to assess qualitative aspects that automated metrics miss, such as readability, modularity, and maintainability (Zheng et al., 2023). These methods provide several insights but suffer from two fundamental limitations. They are not scalable to large codebases, and they lack a standardized evaluation protocol; each study defines its own criteria, making cross-study comparison impossible.

Overall, existing evaluation approaches are misaligned with the requirements of full-stack application assessment. Similarity-based metrics lack cross-file awareness, execution-based benchmarks do not test integrated system behavior, and feedback-based methods lack standardization. None can assess backend-frontend cohesion, database schema consistency, or alignment between generated code and user requirements across architectural layers. These limitations, summarized in Table 2, motivate the development of SFCEval, which combines structural integrity checks with functional coherence assessment across all application layers.

3. Description of the Framework

This section presents SCOPE, an orchestration framework based on prompt-engineering that generates full-stack web applications from natural-language specifications. The framework is organized around two ideas. First, *Extended Self-Planning* decomposes the generation process into three explicit planning stages inspired by the SDLC (Jain and Suman, 2015). Second, a *Division-of-Labor pipeline* chains four modules in a fixed sequence, propagating accumulated context so that each downstream module can reference every upstream artifact.

3.1. Extended Self-Planning

Conventional self-planning (Jiang et al., 2024) generates a single monolithic plan that conflates requirements analysis, architectural design, and task decomposition. Extended Self-Planning addresses this limitation by decomposing planning into three sequential stages, each producing a distinct intermediate representation.

Given a user prompt u, Extended Self-Planning is a composition of three transformations:

$$ESP(u) = (\varphi_{task} \circ \varphi_{arch} \circ \varphi_{req})(u) \tag{1}$$

producing three intermediate representations:

$$R = \varphi_{req}(u) \qquad \text{(Requirements Analysis)} \tag{2}$$

$$P = \varphi_{arch}(u, R) \qquad \text{(Architectural Planning)} \tag{3}$$

$$T = \varphi_{task}(u, R, P) \qquad \text{(Task Decomposition)} \tag{4}$$

where $R = (A, \{r_1, \ldots, r_m\})$ comprises an augmented description A and a set of structured requirements $\{r_1, \ldots, r_m\}$, P is an architectural specification, and $T = \{t_1, \ldots, t_n\}$ is a set of atomic implementation tasks.

Table 2
Comparison of LLM-based frameworks by evaluation coverage and scalability. *Execution-Based Eval.* indicates whether the framework assesses generated code through execution or runtime checks; *Structural Eval.* refers to the presence of explicit structural or architectural evaluation of the generated artifacts; *Scalability* denotes the framework's ability to handle increasing project complexity and size.

Framework	Execution-Based Eval.	Structural Eval.	Scalability
ChatDev (Qian et al., 2024)	yes	Partial	Low
CodeCoR (Pan et al., 2025)	yes	no	Low
SoA (Ishibashi and Nishimura, 2024)	yes	no	High
HiLDe (Gonzalez et al., 2025)	no	no	Low
metaGPT (Hong et al., 2024)	yes	no	Mid
SCOPE (ours)	yes	yes	High

Each stage serves a specific purpose. Requirements Analysis transforms the raw prompt u into a structured representation R. First, u is expanded into an augmented description A that surfaces latent entities, data flows, and service boundaries across architectural layers. Then, u and A are jointly analyzed to extract a set of verifiable, independent requirements $\{r_1, \ldots, r_m\}$. Architectural Planning produces a global specification P that defines system components, entity relationships, and the file-system structure. Task Decomposition breaks P and R into atomic tasks T, each associated with one layer (database, backend, or frontend) and one deliverable artifact. Tasks are generated per layer but with full visibility of all previously generated tasks.

The key difference from conventional self-planning is the *separation of concerns across stages*. Each stage operates on the output of the previous one, so ambiguities in u are resolved in R before structural decisions are made in P. This prevents the conflation identified in Section 2: the model no longer needs to simultaneously interpret requirements and design architecture.

3.2. Pipeline Formalization

The SCOPE pipeline chains four modules in a fixed sequence, each implemented as a set of prompt-conditioned LLM transformations. The pipeline extends Extended Self-Planning with a fourth stage that translates tasks into executable code. The SCOPE pipeline is a sequential composition of four modules:

$$\mathcal{P} : u \xrightarrow{\mathcal{F}_1} R \xrightarrow{\mathcal{F}_2} P \xrightarrow{\mathcal{F}_3} T \xrightarrow{\mathcal{F}_4} C \tag{5}$$

where $\mathcal{F}_1$ is the Prompt Engineering Module (point *(a)* in Fig 1), $\mathcal{F}_2$ is the Project Plan Module (point *(b)* in Fig 1), $\mathcal{F}_3$ is the Task Generation Module (point *(c)* in Fig 1), and $\mathcal{F}_4$ is the Code Generation Module (point *(d)* in Fig 1). Each module $\mathcal{F}_i$ is internally defined as a sequence of prompt-conditioned transformations:

$$\mathcal{F}_i = \left(f_{i,1}^{\pi_{i,1}},\ f_{i,2}^{\pi_{i,2}},\ \ldots,\ f_{i,k_i}^{\pi_{i,k_i}} \right) \tag{6}$$

where each $f_{i,j}^{\pi_{i,j}}$ denotes a single LLM call guided by a prompt configuration $\pi_{i,j}$. The pipeline is designed to enforce four structural properties. These properties are not formally verified at runtime, but are embedded in the prompt design and validated empirically in Section 5.

Context Monotonicity. The context available to each module is non-decreasing. Let $ctx(\mathcal{F}_i)$ denote the set of artifacts accessible to module $\mathcal{F}_i$. Then:

$$ctx(\mathcal{F}_i) \subseteq ctx(\mathcal{F}_{i+1}) \quad \forall\, i \in \{1, 2, 3\} \tag{7}$$

Concretely: $ctx(\mathcal{F}_1) = \{u\}$, $ctx(\mathcal{F}_2) = \{u, A, R\}$, $ctx(\mathcal{F}_3) = \{u, A, R, P\}$, $ctx(\mathcal{F}_4) = \{u, A, R, P, T\}$, where A is the augmented description produced within $\mathcal{F}_1$. This property ensures that no upstream information is lost, mitigating the lack of shared artifacts identified in multi-agent frameworks (Qian et al., 2024; Hong et al., 2024) and multi-phase development.

Architectural Traceability. Every requirement in R is traceable to at least one task in T and at least one code artifact in C. Formally, let $\tau_T : R \to 2^T$ and $\tau_C : R \to 2^C$ be traceability mappings, where 2^T and 2^C are the power sets of T and C respectively. Then:

$$\forall\, r \in R :\ \tau_T(r) \neq \emptyset \ \wedge\ \tau_C(r) \neq \emptyset \tag{8}$$

This property operationalizes requirements traceability within the generation pipeline: no requirement specified in R should be silently dropped during task decomposition or code generation.

Layer Completeness. The generated codebase C contains non-empty components for each architectural layer:

$$C = C_{db} \cup C_{be} \cup C_{fe}, \quad C_{db} \neq \emptyset \ \wedge\ C_{be} \neq \emptyset \ \wedge\ C_{fe} \neq \emptyset \tag{9}$$

where C_{db}, C_{be}, and C_{fe} denote the database, backend, and frontend components, respectively.

Dependency-Ordered Generation. Within each multi-layer module ($\mathcal{F}_3$ and $\mathcal{F}_4$), layer-specific transformations f follow a fixed topological order that mirrors data dependencies in three-tier architectures:

$$f_{db} \prec f_{be} \prec f_{fe} \tag{10}$$

where $\prec$ denotes execution precedence. The backend transformation f_{be} receives the output of f_{db} as input; the frontend transformation f_{fe} receives the outputs of both f_{db} and f_{be}. This ordering ensures that each layer is generated with full visibility of its dependencies.

These four properties collectively address the limitations identified in Section 2. Context Monotonicity prevents information loss across stages. Architectural Traceability prevents requirement dropping. Layer Completeness guarantees full-stack coverage. Dependency-Ordered Generation ensures cross-layer consistency by construction.

3.3. Architecture of SCOPE

Figure 1 illustrates the pipeline architecture. The four modules are executed sequentially, each receiving the accumulated context from all upstream stages. Every prompt configuration $\pi_{i,j}$ in the pipeline follows a standardized structure comprising: (i) a role assignment that defines the agent's persona and objective, (ii) an input/output schema that constrains the format of the expected response, (iii) domain guidelines encoding architectural heuristics (e.g., RESTfulness, SOLID principles), (iv) mandatory and negative constraints that specify what must and must not appear in the output, and (v) failure-prevention instructions (see A). Beyond this shared structure, each prompt includes additional sections specific to its module role. The rest of this section describes each module in terms of its SDLC purpose, inputs and outputs, and prompt-engineering strategies.

3.3.1. Prompt Engineering Module ($\mathcal{F}_1$)

Role and Scope. The Prompt Engineering Module implements the Requirements Analysis stage of Extended Self-Planning (Section 3.1). It receives the raw user prompt u and produces two artifacts: an augmented description A and a structured requirements set $\{r_1, \ldots, r_m\}$, jointly forming $R = (A, \{r_1, \ldots, r_m\})$. Both are propagated to all downstream modules. Formally, the module is defined as a sequence of two prompt-conditioned transformations:

$$\mathcal{F}_1 = \left(f_{1,aug}^{\pi_{1,aug}},\ f_{1,req}^{\pi_{1,req}} \right) \tag{11}$$

where $f_{1,aug}^{\pi_{1,aug}}$ generates the augmented description $A = f_{1,aug}^{\pi_{1,aug}}(u)$ and $f_{1,req}^{\pi_{1,req}}$ extracts the structured requirements $\{r_1, \ldots, r_m\} = f_{1,req}^{\pi_{1,req}}(u, A)$. This separation ensures that the LLM first resolves what u implies (interpretation), then determines what u requires (specification).

Prompt Engineering. Both calls combine ICL and CoT. The first step expands u into a detailed architectural specification within the three-tier template. For instance, one example maps the prompt *"I want a blog where the owner can manage posts and visitors can leave comments"* into a description that identifies: the main domain entities (users, articles, comments, categories) and how they relate to each other, the core functionalities the system must support (user authentication, content management, role-based access, media handling), and the user-facing areas of the application (a public section for reading and commenting, a protected section for content administration). These examples teach the model to infer components that u leaves unstated, such as category management, slug generation,

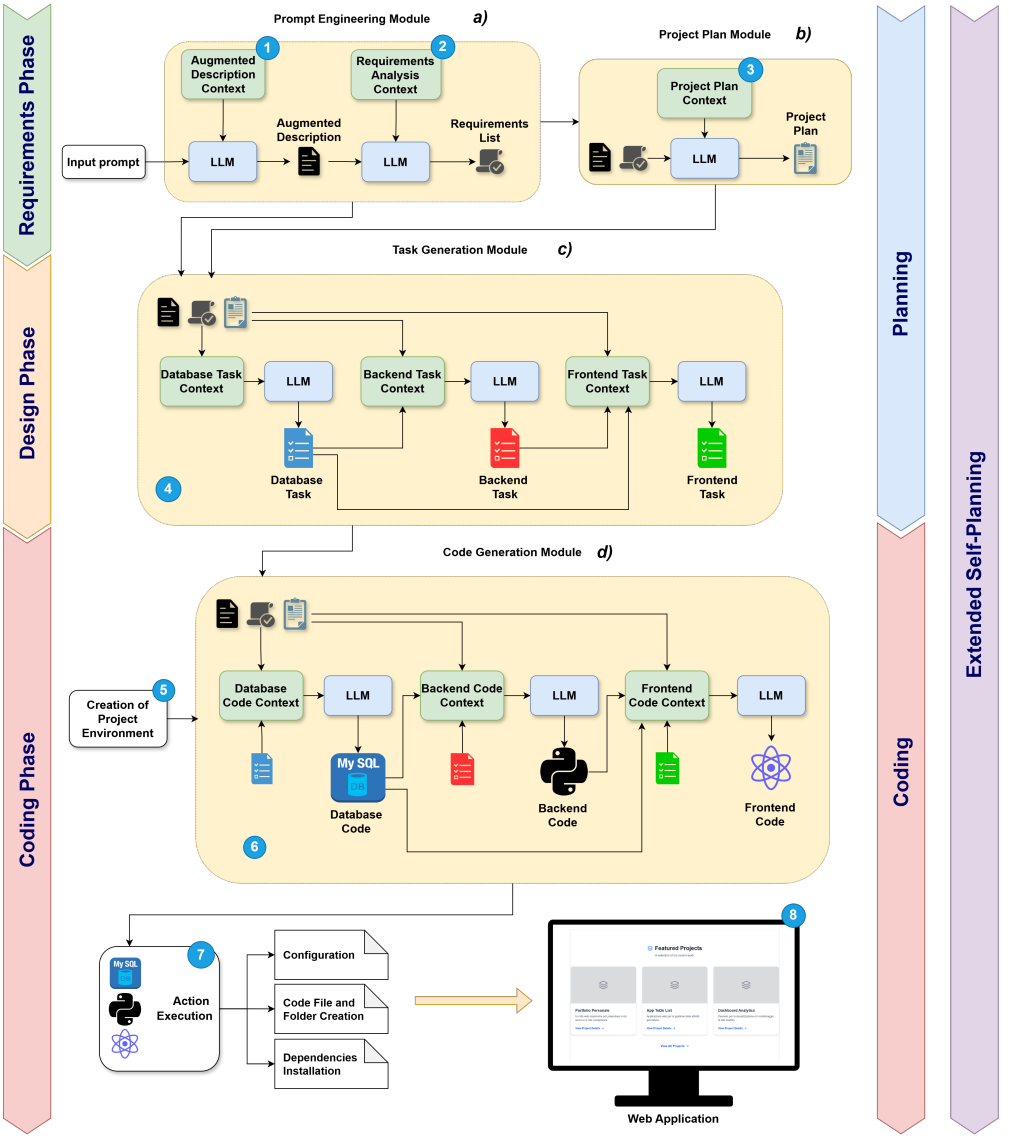

Figure 1: High-level architecture of SCOPE

or session handling. The second call extracts structured requirements from u and A. The prompt integrates established requirements engineering best practices (Jitnah et al., 1995) and enforces them through negative constraints. For example, the model must not produce compound requirements such as "users shall authenticate and view a personalized dashboard," which conflates two independent concerns.

CoT is applied through the prompt structure itself. Each prompt instructs the model to reason step by step: first, identify the application domain, then enumerate the entities involved, then define data flows between layers, and finally produce the structured output. This sequential reasoning reduces the risk of omitting implicit requirements. For instance, when the prompt mentions "blog owner" and "visitors," the CoT structure guides the model to recognize two distinct user roles, which in turn implies role-based access control, a requirement that would likely be missed in a single-step generation.

3.3.2. Project Plan Module ($\mathcal{F}_2$)

Role and Scope. The Project Plan Module implements the Architectural Planning stage of Extended Self-Planning (Section 3.1). It corresponds to the system design phase of the SDLC. The module receives the full output of $\mathcal{F}_1$, namely

$\{u, A, \{r_1, \ldots, r_m\}\}$, and produces a single natural-language architectural specification P. This artifact encodes both strategic guidance (application domain, entity relationships, API design principles) and structural definition (directory hierarchy, file organization, component responsibilities). Unlike the other modules, P is not a structured data object but a free-text document. This design serves two purposes: P acts as an LLM-readable context for downstream modules and as a human-readable document for inspection and debugging. P also addresses a limitation shared by existing multi-agent frameworks (Qian et al., 2024; Hong et al., 2024): the absence of a single authoritative artifact that all downstream agents reference. Formally, the module consists of a single prompt-conditioned transformation:

$$\mathcal{F}_2 = \left(f_{2,plan}^{\pi_{2,plan}} \right) \tag{12}$$

where $P = f_{2,plan}^{\pi_{2,plan}}(u, A, \{r_1, \ldots, r_m\})$.

Prompt Engineering. The call combines ICL and CoT. ICL examples demonstrate how to translate a set of requirements into a coherent architectural blueprint. Each example shows the expected level of specificity. For instance, one example maps the requirements of a content management system into a specification that identifies: the core domain entities and their relationships (e.g., users own articles, articles belong to categories, comments reference both users and articles), the logical grouping of backend responsibilities into dedicated modules (e.g., authentication handling, content management, media processing), and the separation of frontend areas into public-facing pages and protected administration views. These examples anchor the model to a concrete three-tier stack, ensuring that architectural decisions remain grounded in a specific technology baseline.

CoT prompting guides the model through a fixed reasoning sequence. First, define the data model and entity relationships, then design the backend API structure around those entities, then determine the frontend pages that consume the API, and finally specify the file-system layout. This order mirrors the natural dependency chain in three-tier architectures: data structures constrain API design, which in turn constrains frontend behavior. The prompt also enforces consistency with R: the model is instructed to verify that every entity referenced in the requirements appears in the data model, and that every functional requirement maps to at least one backend endpoint and one frontend view. This design supports the Architectural Traceability property at the prompt level.

3.3.3. Task Generation Module ($\mathcal{F}_3$)

Role and Scope. The Task Generation Module implements the Task Decomposition stage of Extended Self-Planning (Section 3.1). It corresponds to the detailed design phase of the SDLC: high-level architectural decisions are translated into concrete work units. The module receives the accumulated context $\{u, A, R, P\}$ and produces a task set $T = T_{db} \cup T_{be} \cup T_{fe}$, where each subset contains layer-specific tasks. Each task $t_i \in T$ specifies a natural-language description of the deliverable, a target file path within the project structure defined in P, and a layer label $\ell(t_i) \in \{db, be, fe\}$. This stage bridges design and implementation: it decomposes P into atomic units that the Code Generation Module can execute independently. Formally, the module is defined as a sequence of three prompt-conditioned transformations:

$$\mathcal{F}_3 = \left(f_{3,db}^{\pi_{3,db}},\ f_{3,be}^{\pi_{3,be}},\ f_{3,fe}^{\pi_{3,fe}} \right) \tag{13}$$

where $T_{db} = f_{3,db}^{\pi_{3,db}}(u, A, R, P)$, $T_{be} = f_{3,be}^{\pi_{3,be}}(u, A, R, P, T_{db})$, and finally $T_{fe} = f_{3,fe}^{\pi_{3,fe}}(u, A, R, P, T_{db}, T_{be})$. This ordering follows the Dependency-Ordered Generation property: each layer receives the tasks of its dependencies as additional context.

Prompt Engineering. All three calls combine ICL and CoT. ICL examples show how to decompose an architectural plan into granular tasks. They enforce cross-layer referential consistency. For instance, one example maps a database task defining a user entity to a backend task that creates an authentication module referencing that entity. The same example produces a frontend task that implements a login view and consumes the authentication endpoint. These demonstrations teach the model that no task may reference entities or components that have not yet been defined.

CoT structures the reasoning within each layer call. For the backend, the model follows four steps: review database tasks to identify available entities, determine which functionalities each entity requires, group functionalities into modules following P, and emit one task per module. The frontend call follows an analogous sequence over available endpoints. This decomposition prevents references to non-existent entities and duplication across modules. When a layer exceeds the context window, the module issues multiple calls and concatenates the results, improving the scalability.

3.3.4. Code Generation Module ($\mathcal{F}_4$)

Role and Scope. The Code Generation Module is the implementation stage of the pipeline. It corresponds to the coding phase of the SDLC (Jain and Suman, 2015). The module receives the maximal accumulated context $\{u, A, R, P, T\}$. Additionally, each layer-specific call receives the code generated by prior layers. The output is the complete codebase $C = C_{db} \cup C_{be} \cup C_{fe}$. It is the only module where all three prompting paradigms, ICL, CoT, and CoC (Li et al., 2024b), are applied simultaneously. Before generation begins, SCOPE initializes the project environment. It creates the directory hierarchy defined in P and bootstraps the frontend with a pre-configured Next.js project. Each call produces a structured JSON output containing the source code, the target file path, and a component label. After all layers are generated, SCOPE writes each file to its designated path and generates dependency specifications for automated environment setup. Formally:

$$\mathcal{F}_4 = \left(f_{4,db}^{\pi_{4,db}},\ f_{4,be}^{\pi_{4,be}},\ f_{4,fe}^{\pi_{4,fe}} \right) \tag{14}$$

where $C_{db} = f_{4,db}^{\pi_{4,db}}(u, A, R, P, T)$, $C_{be} = f_{4,be}^{\pi_{4,be}}(u, A, R, P, T, C_{db})$, and $C_{fe} = f_{4,fe}^{\pi_{4,fe}}(u, A, R, P, T, C_{db}, C_{be})$. This ordering instantiates the Dependency-Ordered Generation property. The result is a complete full-stack web application.

Prompt Engineering. All three calls combine ICL, CoT, and CoC. ICL examples are tailored to a standardized technology stack: MySQL for the database, Flask for the backend, and Next.js, React.js and Tailwind.css for the frontend. Each example demonstrates expected code structure, naming conventions, and import patterns. For instance, one backend example shows how the authentication module imports the user entity from the database layer, defines route handlers using the Flask Blueprint pattern, and validates requests using typed schemas. The corresponding frontend example shows a login component that calls the authentication endpoint, handles the response, and stores the session state. These examples anchor the model to concrete implementation patterns.

CoT structures the reasoning before code emission. For each task, the model follows four steps: review the task description and identify dependencies; determine required imports from the technology stack and prior code; plan function signatures and data flow; and generate the implementation. This chain is critical to both the backend and the frontend. For example, when generating a component that displays articles, the model must first identify the relevant endpoint from C_{be}, determine its response schema, and generate the corresponding data-fetching and rendering logic only then.

CoC prompting (Li et al., 2024b) is applied exclusively in this module (view Fig 2). It encourages the model to interleave natural-language reasoning with executable code constructs. For instance, when implementing a database initialization script, the model first writes a pseudocode outline: create engine, define base model, create tables, seed initial data. It then expands each step into executable code. This intermediate reasoning step reduces misalignment between the task description and the generated implementation. It is particularly effective for complex tasks involving multiple interacting components.

3.4. Discussion

The pipeline uses ICL, CoT, and CoC rather than alternative paradigms such as fine-tuning or reinforcement learning. Prompt-based techniques require no model retraining and allow each module to encode domain-specific heuristics directly in the prompt configuration. Tree-of-Thought prompting (Yao et al., 2023) was excluded due to its branching strategy: it would multiply LLM calls per module and significantly increase generation latency. Self-refinement loops (Zhang et al., 2023; Olausson et al., 2024; Chen et al., 2024c; Madaan et al., 2023; Chang et al., 2023) were excluded for similar reasons: iterative revision cycles would compound latency across four sequential modules without guaranteeing convergence.

This design directly addresses the limitations identified in Sections 1 and 2. Existing pipelines either collapse the development lifecycle into a single generation step or rely on local repair mechanisms without cross-component visibility (Jiang et al., 2024; Madaan et al., 2023; Zhang et al., 2023). SCOPE mitigates lifecycle conflation through Extended Self-Planning, which explicitly separates requirements analysis, architectural design, and task decomposition. The shared artifact P provides a single architectural reference that all downstream modules consume, addressing the absence of constrained inter-stage communication in multi-agent frameworks (Qian et al., 2024; Hong et al., 2024). The structured elicitation in $\mathcal{F}_1$ mitigates sensitivity to ambiguous user prompts by surfacing implicit requirements before any architectural decision is made. Dependency-ordered generation reduces the risk of cross-layer inconsistencies by ensuring that each layer is generated with full visibility of its dependencies.

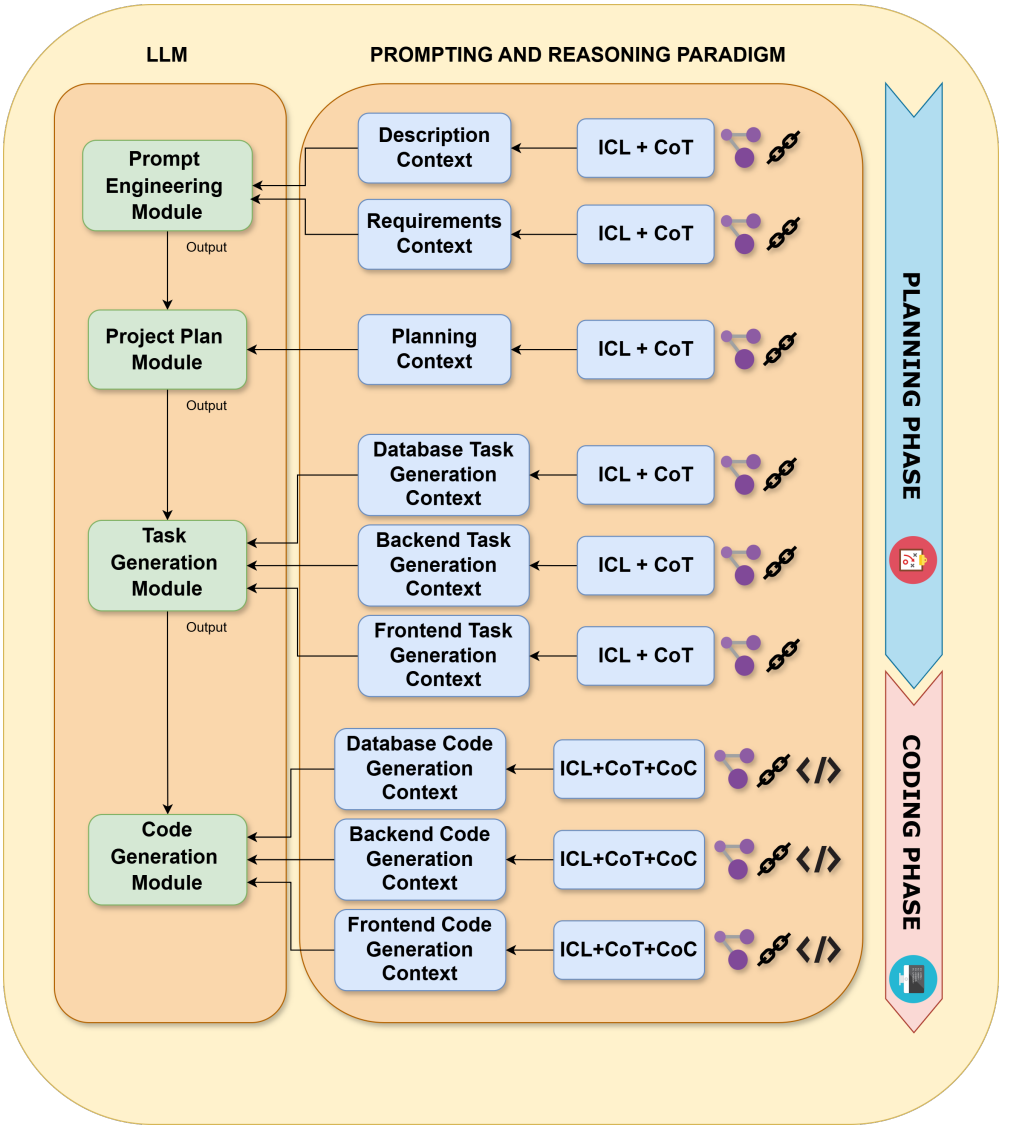

Figure 2: Overview of the prompting paradigms applied across the framework pipeline

These design decisions collectively position SCOPE at a different level of abstraction than existing frameworks. Where systems such as ChatDev, CodeCoR, and MetaGPT target individual components or narrowly scoped programs, SCOPE generates a complete three-tier application whose layers must be mutually consistent. This difference is not merely quantitative (more files) but qualitative: the output is an integrated system whose correctness depends on cross-layer properties, schema-to-API alignment, and endpoint-to-component binding that do not arise in single-component generation. As a consequence, the evaluation methodology must assess system-level integrity rather than file-level correctness alone. Section 4 introduces an evaluation design that reflects this scope, and Section 5 empirically evaluates whether the design choices described above translate into measurable improvements in structural consistency, functional correctness, and requirements coverage.

4. Evaluation Methodology

To rigorously evaluate the SCOPE framework's capabilities for automating full-stack web application development, we designed a comprehensive experimental study. This section details the research questions, the experimental setup, and SFCEval, our proposed methodology for quantifying the structural and functional integrity of generated software.

4.1. Research Questions

The evaluation is guided by three main Research Questions (RQs):

- RQ1 (Effectiveness and Consistency): To what extent is SCOPE capable of generating syntactically correct, executable, and functionally consistent web applications that maintain coherence across the database, backend, and frontend layers?
- RQ2 (Module Contribution): What is the quantitative impact of the Prompt Engineering Module on the overall quality and semantic alignment of the generated software?
- RQ3 (Model Robustness): How does the framework's performance vary when different LLMs are employed as the core generative engines within the pipeline?

4.2. Experimental Setup

To address the research questions, we designed an experimental protocol centered on a realistic case study. The target application is a "Machine Learning Engineer Blog" platform, selected because it exercises all three architectural layers: a relational database with multiple entities and relationships, a backend API with authentication and CRUD operations, and a responsive frontend with both public and protected views. The evaluation targets a single domain to enable a controlled comparison across different LLMs on the same generation task.

Prompt Variations. Five prompt versions were designed to assess SCOPE's sensitivity to varying levels of input specificity (see B). Each version corresponds to a distinct specificity level: Low, Low-Mid, Mid, Mid-High, and High. The Low prompt omits most functional and architectural information. The Low-Mid and Mid prompts progressively introduce partial requirements and structural constraints. The Mid-High and High prompts specify explicit functional, structural, and stylistic constraints, including UI components and layout behavior. This graduated design enables a controlled analysis of how prompt completeness affects generation quality.

Scope of the Empirical Comparison. The evaluation compares multiple LLMs operating within the SCOPE pipeline rather than comparing SCOPE against alternative generation frameworks. This design reflects two constraints. First, as established in Section 2.2, no existing framework provides a complete three-tier web application (database, backend, and frontend) as an integrated system. A comparison would therefore require evaluating fundamentally different output types, isolated functions or single-layer components versus full-stack applications, under the same metrics, which would not yield meaningful conclusions. Second, the evaluation methodology introduced in Section 4.3 is specifically designed to assess system-level properties, including cross-layer communication, schema consistency, and end-to-end requirement satisfaction. These properties are undefined for single-component outputs and cannot be retroactively imposed on frameworks that do not target full-stack generation. The cross-model comparison within the pipeline (Section 4.5) serves a different but equally important purpose: it isolates the orchestration framework's contribution from the capabilities of any single LLM, demonstrating that the pipeline's structural properties hold across different generative engines.

4.3. Evaluation Metrics

The generated applications are evaluated along two complementary dimensions. First, Execution Accuracy (EA) (Chen et al., 2024b) provides a widely adopted baseline measure in the literature. It verifies that generated files are syntactically valid and runnable, but operates at the file level without cross-file awareness. As discussed in Section 2, file-level metrics cannot detect architectural inconsistencies between layers. To address this gap, we introduce SFCEval, a multi-dimensional methodology that assesses structural integrity, cross-layer coherence, and functional alignment with user requirements. By applying both metrics to the same generated applications, we can directly quantify the gap between file-level correctness and system-level quality.

4.3.1. Execution Accuracy

Execution Accuracy (EA) (Chen et al., 2024b) measures the fraction of generated source files that can be parsed and executed without syntax or runtime errors. Let N_{total} be the total number of generated files and N_{exec} the number that execute successfully. The metric is defined as:

$$EA = \frac{N_{exec}}{N_{total}} \tag{15}$$

Backend files are validated by running the Python interpreter, database scripts by executing the schema against a MySQL instance, and frontend files are validated through the Next.js build process.

4.3.2. SFCEval: A Multi-Dimensional Assessment Framework

SFCEval evaluates generated applications along two dimensions. *Structural and syntactic integrity* verifies that the codebase compiles, dependencies resolve correctly, imports are valid, and the database schema is well-formed. *Functional consistency* assesses cross-layer coherence and alignment with user requirements.

Structural and Syntactic Integrity. This category evaluates the technical correctness of the codebase and the database structure. It verifies that the software compiles, that the environment is reproducible, and that the fundamental data architecture (the database) is sound.

- *Syntax Integrity:* This metric measures the absence of syntax errors across the entire codebase. They do not indicate logical flaws, but they completely prevent the software from executing. Let N_{syn} be the number of syntactically correct files:
$$SI = \frac{N_{syn}}{N_{total}}$$
- *Dependencies Installation:* This metric verifies that all external libraries are correctly listed in configuration files (e.g., requirements.txt). If some dependencies are missing or misconfigured, the system may fail to execute, but such issues are generally straightforward to identify and fix. Let δ be a binary indicator where $\delta = 1$ if all dependencies install successfully, and $\delta = 0$ otherwise:
$$DI = \delta$$
- *Import Correctness:* This metric validates that imported modules exist and are syntactically accessible. It is evaluated both for the backend and the frontend. Let N_{imp}^{corr} be the number of valid imports and N_{imp}^{total} be the total imports found:
$$IC = \frac{N_{imp}^{corr}}{N_{imp}^{total}}$$
- *Database Creation:* assesses the consistency of the SQL schema and ORM models by verifying primary keys, foreign keys, and data type validity. Schema defects can propagate across all layers, causing data loss or preventing CRUD operations. To tolerate minor generation defects, we introduce a threshold $T = 10$, calibrated on the average schema size empirical observed across the generated applications. If the number of errors N_{err} exceeds T, the database is deemed unsound:
$$DC = \max\left(0,\ \frac{T - N_{err}}{T}\right) \tag{16}$$

Functional Consistency and Operational Validity. This category assesses the application's semantic quality, focusing on layer integration and adherence to business logic.

- Frontend-Backend Communication: This measures the reliability of the integration between the frontend and backend layers. A failure in this communication represents a serious functional defect, as it renders part of the app unusable. It calculates the average of *Coverage* (C, percentage of backend endpoints actually utilized by the frontend) and *Robustness* (R, presence of error handling logic for those calls):
$$FBC = \frac{C + R}{2}$$

- Functional Assessment: For each prompt, a set of functional ground truths is derived from the original prompt u before any application is inspected. Each ground truth follows the format: precondition → user action → expected system state. Each requirement is then verified through a strict boolean protocol: a requirement is satisfied if and only if the expected system state is observed. Let N_{req}^{sat} be the number of satisfied requirements and N_{req}^{total} the total:

$$FA = \frac{N_{req}^{sat}}{N_{req}^{total}}$$

Table 3 shows an example of the steps to follow to ensure that a functional requirement is met.

Table 3
Example of a functional assessment procedure in SFCEval for the requirement: *"As a user, I want to be able to add a skill to my blog, so that I can present my competencies to visitors."*

Step	Precondition	Action	Expected State
1	User is authenticated	Click *Add Skill*	Form is displayed
2	Form is displayed	Fill and submit the form	Skill is stored
3	Skill is stored	Navigate to public profile	Skill is visible

Weighting Strategy. SFCEval aggregates the six metrics into a single normalized score $(0, 1)$ through a weighted sum. The weight w_i assigned to each metric reflects the severity of the corresponding defect type, assessed along three dimensions: remediation time, technical complexity, and functional relevance. This approach follows established practice in software quality assessment, where metrics of different granularity are aggregated through severity-aware weighting (Ulan et al., 2021). Weights were assigned by the authors following the Goal-Question-Metric paradigm (Caldiera and Rombach, 1994). The goal is to evaluate whether the generated application is functionally consistent and correct, which prioritizes metrics that capture system-level behavior over those that capture file-level defects. Accordingly, syntax integrity ($w = 0.15$), dependency installation ($w = 0.10$), and import correctness ($w = 0.10$) receive lower weights, as they represent easily detectable and fixable issues. Database creation ($w = 0.20$) and frontend-backend communication ($w = 0.20$) receive higher weights, as they capture cross-layer consistency. Functional assessment receives the highest weight ($w = 0.25$), as requirement satisfaction determines the overall success of the application. The final SFCEval score is:

$$SFCEval = \sum_{i=1}^{6} w_i \cdot m_i \tag{17}$$

Table 4 reports the complete weight assignment.

Table 4
Evaluation tasks and corresponding weights in SFCEval

Evaluation Task	**Weight**
Syntax Integrity	0.15
Dependencies Installation	0.10
Import Correctness	0.10
Database Creation	0.20
Frontend-Backend Communication	0.20
Functional Assessment	0.25
Total	**1.00**

4.4. Ablation Study Methodology

The ablation study isolates the contribution of the Prompt Engineering Module ($\mathcal{F}_1$) by removing it from the pipeline and routing the raw user prompt u directly to the Project Plan Module ($\mathcal{F}_2$). In this configuration, $\mathcal{F}_2$ receives u

as its sole input, without the augmented description A or the structured requirements $\{r_1, \ldots, r_m\}$ that $\mathcal{F}_1$ would normally produce. All other modules ($\mathcal{F}_2$, $\mathcal{F}_3$, $\mathcal{F}_4$) retain their standard prompt configurations and execute in their original sequence. The resulting applications are compared against the complete-pipeline outputs using the SFCEval metrics defined in Section 4.3.2.

Choice of Ablation Target. The pipeline's context-accumulating design (Section 3.2) creates an asymmetry among modules with respect to independent removability. $\mathcal{F}_2$ produces the architectural specification P, which serves as the mandatory input for task decomposition in $\mathcal{F}_3$. Removing $\mathcal{F}_2$ would leave $\mathcal{F}_3$ without an artifact to decompose, reducing its input to the raw prompt and the requirements set, an input for which $\mathcal{F}_3$'s prompts are neither designed nor calibrated. Similarly, $\mathcal{F}_3$ produces the task set T that $\mathcal{F}_4$ requires to structure code generation. In both cases, removal would not produce a meaningfully degraded pipeline but a fundamentally broken one, yielding outputs that cannot be assessed with SFCEval or any other structured evaluation methodology.

$\mathcal{F}_1$ occupies a structurally distinct position. Its outputs enrich the downstream context, surfacing implicit requirements, resolving ambiguities, and providing a refined specification, but the downstream modules can still operate on the raw prompt u, which remains a valid, if impoverished, input. This makes $\mathcal{F}_1$ the only module whose removal produces a *degraded but functional* pipeline: one that generates complete applications of reduced quality rather than incomplete or incoherent artifacts. The ablation therefore quantifies $\mathcal{F}_1$'s marginal contribution to generation quality under controlled conditions.

4.5. Comparative Model Analysis

The pipeline was executed using three LLMs: OpenAI o3-mini-high, Google Gemini 2.5 Pro, and DeepSeek V3 0324. All models received identical inputs: the same five prompts, the same system instructions, and the same ICL examples. API parameters were set to default values across all models to ensure a fair comparison. O3-mini-high was additionally run twice per prompt to assess generation stability and reproducibility, yielding 10 applications. Gemini 2.5 Pro and DeepSeek V3 were run once per prompt, yielding 5 applications each. Five additional applications were generated with o3-mini-high for the ablation study described in Section 4.4. In total, 25 full-stack web applications were generated and evaluated.

5. Results and Discussion

5.1. Results and Discussion of the applied metrics

To address RQ1, the generated web applications were evaluated using two complementary metrics: Execution Accuracy and SFCEval. SFCEval alone is sufficient to answer RQ1, as its six metrics jointly assess syntactic correctness, structural integrity, and functional conformance across all architectural layers. EA was additionally adopted as a file-level executability baseline to quantify the gap that SFCEval is designed to fill. EA verifies whether files run without errors, but cannot detect semantic defects, missing functionalities, or cross-layer inconsistencies that SFCEval captures.

Evaluation Protocol. All SFCEval assessments were carried out by a single evaluator following the structured protocol defined in Section 4.3.2. To mitigate subjectivity, the protocol was designed to minimize discretionary judgment at every step. Structural metrics (Syntax Integrity, Dependency Installation, Import Correctness, Database Creation) are verified through deterministic, tool-assisted checks: the Python interpreter, the MySQL engine, the Next.js build process, and static import resolution, respectively. These metrics leave no room for subjective interpretation. Frontend–Backend Communication is assessed by cross-referencing the set of declared backend endpoints against the frontend source code, yielding a coverage ratio and a binary check for error-handling logic. Functional Assessment, the most judgment-sensitive metric, follows a strict boolean protocol: each requirement is decomposed into a precondition–action–expected state triple (see Table 3), and a requirement is marked as satisfied if and only if the expected state is observed after executing the prescribed action sequence.

5.1.1. Execution Accuracy Results

Table 5 reports EA scores across all models and prompt specificity levels. O3-mini-high achieves the highest mean EA (0.958 and 0.944 across two independent runs), indicating that the large majority of generated files compile and execute without errors. The difference between the two runs remains below 0.02, suggesting low generation variance, although the limited number of runs prevents definitive claims about stability.

Table 5
Execution Accuracy results across multiple LLMs and prompt specificity levels.

	o3-mini-high		Gemini 2.5 Pro	DeepSeek V3
Prompt Specificity	**Run 1**	**Run 2**		
Low	0.96	0.94	**0.92**	**0.95**
Low-Mid	0.95	**0.97**	0.91	0.94
Mid	0.93	0.90	0.89	0.92
Mid-High	**0.98**	0.95	0.87	0.90
High	0.97	0.96	0.85	0.89
Mean	**0.958**	**0.944**	**0.888**	**0.920**

Gemini 2.5 Pro obtains a lower mean EA of 0.888. Manual inspection reveals that this model exhibits difficulties in resolving frontend import paths, generating unresolved or incorrectly specified module references across React components. Because Gemini 2.5 Pro also produces substantially more files per application than the other two models, the import resolution issue propagates across a larger codebase, amplifying its impact on the final EA score. This case illustrates a structural limitation of EA. A model generating semantically correct and well-architected code can score lower than a model producing fewer files.

DeepSeek V3 achieves an intermediate mean EA of 0.920. Manual inspection reveals recurring issues in backend code generation, such as incorrect Flask route registrations, Python indentation errors, and missing `__init__.py` files in subpackages that break module resolution. These defects are individually trivial and easily fixable, yet they accumulate across the generated codebase, reducing the overall EA score.

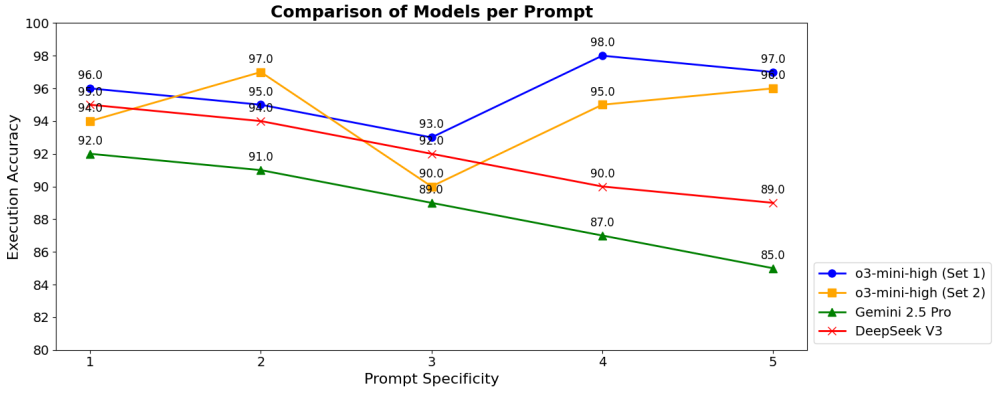

Figure 3: Execution Accuracy as a function of prompt specificity for each LLM evaluated within the SCOPE framework.

Figure 3 reveals an inverse trend between models. For o3-mini-high, EA increases with prompt specificity, allowing the model to generate more precise file structures and imports. Conversely, Gemini 2.5 Pro and DeepSeek V3 exhibit decreasing EA as specificity grows. A plausible explanation is that higher-detail prompts induce these models to generate more granular codebases with additional files and components, amplifying the surface area for cross-file errors.

These results confirm that EA captures file-level executability but remains insensitive to semantic correctness, missing functionalities, and cross-layer consistency. The following section presents SFCEval results, which address these dimensions.

5.1.2. SFCEval Results

Tables 6 and 7 summarize the evaluation results for two independent generations performed with the *o3-mini-high* model for each of the five prompts. Since only two generation runs were performed per prompt, the following observations are descriptive and do not support inferential claims about generation stability. The results reveal a clear

relationship between prompt specificity and generation stability: less detailed prompts lead to discrepancies between generations, whereas more specific prompts yield increasingly consistent outputs.

Table 6
SFCEval metrics scores for the first generation cycle.

Metric/Specificity	Low	Low-Mid	Mid	Mid-High	High
Import Correctness	0.091	0.095	0.100	0.097	0.089
Communication	0.150	0.125	0.185	0.200	0.200
Dependencies	0.100	0.100	0.100	0.100	0.100
Functional Assessment	0.150	0.125	0.146	0.225	0.250
Syntax Integrity	0.130	0.150	0.146	0.146	0.140
Database Creation	0.160	0.200	0.200	0.200	0.200
Total	**0.781**	**0.820**	**0.877**	**0.968**	**0.979**

Table 7
SFCEval metrics scores for the second generation cycle.

Metric/Specificity	Low	Low-Mid	Mid	Mid-High	High
Import Correctness	0.079	0.100	0.093	0.081	0.088
Communication	0.184	0.186	0.167	0.164	0.186
Dependencies	0.100	0.100	0.100	0.100	0.100
Functional Assessment	0.179	0.208	0.125	0.150	0.212
Syntax Integrity	0.130	0.146	0.150	0.150	0.142
Database Creation	0.200	0.200	0.200	0.200	0.200
Total	**0.872**	**0.940**	**0.835**	**0.845**	**0.928**

Per-metric Analysis. Overall, the first dimension of SFCEval, *Structural Consistency*, captures syntactic and architectural integrity across the generated codebase. Dependency installation was always correct across all runs, and Database Creation consistently achieved a near-maximum score in both generations, confirming strong reliability in managing foundational architectural components. The results also show stable performance in Syntax Integrity, with only minimal variation across prompts, indicating that the generated code maintains structural correctness regardless of prompt detail. Import Correctness presents small, expected fluctuations, but values remain consistently positive across all prompts and generations.

Regarding the second dimension, *Functional Consistency*, Communication between layers performs well overall. While its scores vary slightly with lower-specificity prompts, the values remain consistently solid, and the trend shows clear improvement as prompts become more detailed. This confirms that additional contextual detail supports stronger cross-layer coordination. Finally, the Functional Assessment confirms a progressive improvement pattern. Lower-specificity prompts yield lower FA scores because the model must infer implicit requirements left unspecified by the prompt. As the prompt detail increases, FA generally improves. This variance highlights that functional correctness is the most sensitive dimension to both prompt quality and stochasticity in generation.

Validation of Framework Properties. The results obtained for Database Creation, Import Correctness, and Communication provide empirical support for the four properties defined in Section 3.2. These three metrics jointly assess whether each pipeline stage produces well-formed artifacts, whether cross-file references remain valid across modules, and whether backend and frontend components are correctly aligned. The consistently high scores across all configurations confirm that the generated applications satisfy Context Monotonicity, Dependency-Ordered Generation, Architectural Traceability, and Layer Completeness as formalized in Section 3.2.

Answer to RQ1: SCOPE generates full-stack web applications with mean SFCEval scores between 0.781 and 0.979 for o3-mini-high. These results indicate that the framework produces structurally sound applications whose functional completeness scales with input specificity, while confirming that system-level evaluation requires multi-dimensional metrics beyond file-level executability.

5.2. Ablation Study Results

To address RQ2, five web applications were generated without the Prompt Engineering Module ($\mathcal{F}_1$) using o3-mini-high, and evaluated with SFCEval under the same conditions applied to the complete pipeline.

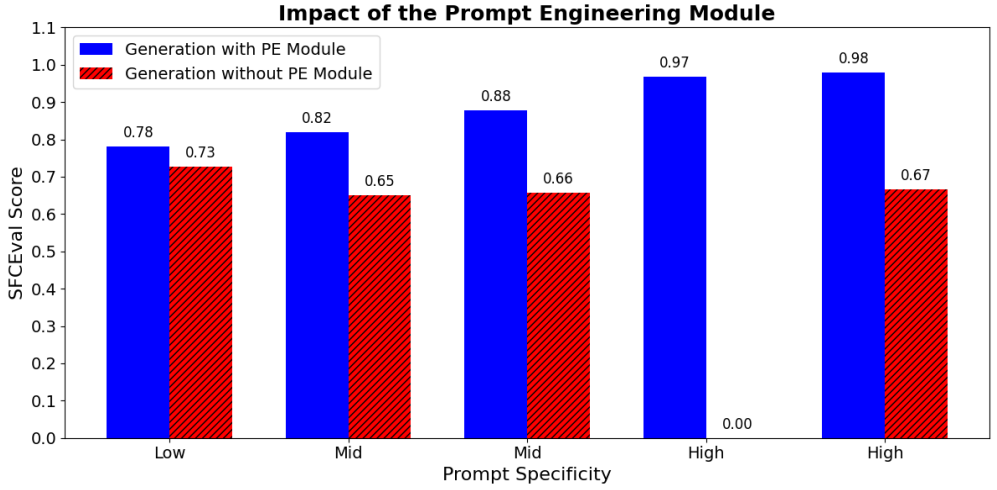

Figure 4: Per-metric SFCEval comparison between the average scores of the two complete-pipeline generations (o3-mini-high) and the ablation runs without $\mathcal{F}_1$, across five prompts of increasing specificity.

Figure 4 shows a consistent degradation across all prompts when $\mathcal{F}_1$ is removed. The average total SFCEval score drops from 0.885 (complete pipeline) to 0.542 (ablation), corresponding to a mean decrease of 0.343 points. Manual inspection of the ablation outputs revealed that the metrics most affected are Functional Assessment and Communication, which exhibit the largest absolute drops. FA decreases because, without the structured requirements refinement performed by $\mathcal{F}_1$, the model fails to capture both explicit and implicit requirements and produces incomplete or placeholder implementations. Communication degrades because the absence of a refined requirement set propagates ambiguity into the architectural plan P, weakening the alignment between backend endpoints and frontend data bindings. Structural metrics (SI, IC, DC) are less affected, indicating that the model retains the ability to generate syntactically valid code even without $\mathcal{F}_1$, but loses the capacity to ensure functional and cross-layer coherence. This pattern is consistent with the asymmetry described in Section 4.4: because $\mathcal{F}_1$ enriches the downstream context rather than structurally constraining it, its removal degrades the semantic quality of the generated artifacts, primarily functional completeness and cross-layer alignment, while leaving the pipeline's ability to produce syntactically valid code largely intact.

In one isolated case, the generated application was deemed non-classifiable: core components were missing and the project structure was incoherent, preventing any SFCEval assessment. Although this is a single outlier, it illustrates the risk of severe architectural breakdown when raw prompts bypass the refinement stage.

These results also relate to the framework properties defined in Section 3.2. The degradation of Communication and FA when $\mathcal{F}_1$ is removed indicates that Context Monotonicity and Architectural Traceability depend on the quality of the initial requirement analysis. If $\mathcal{F}_1$ produces an incomplete or ambiguous requirement set, downstream modules propagate these deficiencies rather than correcting them.

Answer to RQ2. Removing $\mathcal{F}_1$ reduces the mean SFCEval score by 0.343 points, with the largest impact on functional metrics. Although the absolute SFCEval score varies with prompt specificity, the performance gap between the complete pipeline and the ablation configuration remains consistent across all prompt levels. This indicates that $\mathcal{F}_1$ provides a systematic contribution to generation quality that is independent of input detail, confirming its role as a necessary component within the SCOPE pipeline.

5.3. Comparison Results

Table 8 reports the overall SFCEval scores for the three evaluated LLMs across all prompt specificity levels. O3-mini-high achieves the highest mean score (0.885), followed by Gemini 2.5 Pro (0.845) and DeepSeek V3 (0.793).

Table 8
Overall SFCEval scores for the compared LLMs.

Prompt Specificity	o3-mini-high	Gemini 2.5 Pro	DeepSeek V3
Low	0.781	**0.878**	0.809
Low-Mid	0.820	**0.897**	0.789
Mid	**0.877**	0.848	0.710
Mid-High	**0.968**	0.825	0.837
High	**0.979**	0.775	0.822
Mean Value	**0.885**	**0.845**	**0.793**

Cross-model Trend Analysis. The most notable finding is the inverse relationship between prompt specificity and model performance. O3-mini-high increases monotonically from 0.781 (Low) to 0.979 (High), confirming that the pipeline leverages additional input details to produce more complete and coherent applications. Gemini 2.5 Pro exhibits the opposite trend: it achieves the highest score at low-mid specificity (0.897) but drops to 0.775 at high specificity. The evaluation reveals that this degradation is driven by a substantial increase in frontend import-resolution errors as prompt detail increases. Higher-specificity prompts lead Gemini to generate more modularized codebases with more files. This amplifies the import correctness failures, which disproportionately penalize the overall SFCEval score. DeepSeek V3 shows no clear monotonic trend, with scores fluctuating between 0.710 and 0.837 without consistent directionality.

Comparing these trends with the EA results reveals a ranking inversion: DeepSeek V3 outperforms Gemini 2.5 Pro on EA (0.920 vs. 0.888), whereas Gemini leads on SFCEval (0.845 vs. 0.793). This discrepancy confirms that file-level executability does not reflect system-level quality, validating the need for multi-dimensional evaluation.

Table 9
Top-performing LLMs by SFCEval metric

Task	LLM	Mean Score
Imports Correctness	o3-mini-high	0.091
Comm. frontend-backend	Gemini 2.5 Pro	0.187
Correct installation of dependencies	o3-mini-high	0.100
Functional assessment	Gemini 2.5 Pro	0.206
Presence of syntax errors	o3-mini-high	0.142
Correct database creation	Gemini 2.5 Pro	0.200

Table 9 shows that no single LLM achieves the highest score across all metrics. O3-mini-high leads on the structural dimension (Import Correctness, Dependency Installation, Syntax Integrity), while Gemini 2.5 Pro leads on the functional dimension (Communication, Functional Assessment, Database Creation). This complementarity indicates that structural precision and functional completeness are governed by distinct generation capabilities, suggesting that model architecture and training data composition influence each dimension differently.

Answer to RQ3. All three LLMs produce functional web applications within the SCOPE pipeline. However, each model exhibits distinct performance profiles. The inverse trend between these models indicates that framework performance is sensitive to the interaction between LLM characteristics and prompt design.

5.4. Summary

The results presented in this section provide empirical evidence that the three gaps identified in Sections 1 and 2 have been mitigated. The dependency-ordered pipeline produces cross-layer consistent applications, as confirmed by the stable Backend-Frontend Communication and Database Creation scores across all configurations. The Prompt Engineering Module mitigates sensitivity to vague requirements, with the ablation study showing a systematic degradation when structured refinement is removed. SFCEval fills the absence of system-level evaluation methodologies, detecting functional and structural defects that file-level metrics such as EA fail to capture.

6. Conclusion, Limitations, and Future Work

Conclusion. This paper presented SCOPE, a structured code orchestration framework for full-stack web application generation, and SFCEval, a multi-dimensional methodology for evaluating AI-generated software at system level. SCOPE introduces Extended Self-Planning within a pipeline-based architecture that enforces four structural properties: Context Monotonicity, Dependency-Ordered Generation, Architectural Traceability, and Layer Completeness. The evaluation confirmed that SCOPE generates applications with SFCEval scores up to 0.979 (RQ1), that the Prompt Engineering Module provides a systematic contribution independently of prompt specificity (RQ2), and that each LLM exhibits distinct performance profiles with mean scores between 0.793 and 0.885 (RQ3). SFCEval captures structural and functional defects that file-level metrics cannot detect, as demonstrated by the ranking inversion between EA and SFCEval across models. These results address the three gaps identified in Section 1: SCOPE mitigates the lack of cross-layer architectural consistency through dependency-ordered generation, the Prompt Engineering Module reduces sensitivity to vague requirements through structured refinement, and SFCEval fills the absence of system-level evaluation methodologies for AI-generated web applications.

Limitations and Future Work. Although the Prompt Engineering Module substantially reduces issues related to ambiguous requirements, some limitations remain. The positive correlation between prompt specificity and SFCEval scores indicates that the module has not yet fully exploited the available input detail, leaving considerable room for further refinement of the structured elicitation process. The current implementation relies on ICL examples tailored to a specific application stack, which constrains reliable generation to this technological setting. A promising direction for future work is to integrate retrieval-based techniques to dynamically select or construct in-context examples aligned with the target application stack. Moreover, the framework does not yet generate fully deployable applications: human testing is still required to ensure reliability and guarantee production-level robustness. Future work will focus on expanding coverage of contextual learning, integrating automated testing modules, and moving toward end-to-end deployable pipelines that further reduce human intervention. A further limitation concerns the evaluation procedure. All SFCEval assessments were performed by a single evaluator. Although the structured, boolean-based protocol described in Section 5 substantially constrains evaluator discretion. Future iterations will explore the automation of the evaluation process through deep learning and machine learning algorithms trained on the annotated data collected in this study, incorporating security-oriented metrics to assess vulnerability exposure in the generated codebases.

Data availability

https://github.com/matteo270303/SCOPE-Examples

CRediT authorship contribution statement

First Author: Conceptualization, Methodology, Software, Writing – original draft, Writing – review & editing. **Second Author:** Methodology, Writing – review & editing. **Third Author:** Supervision, Validation, Writing – review & editing. **Last Author:** Supervision, Validation.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This research is partially funded by the PNRR project FAIR - Future AI Research (PE00000013), Spoke 6 - Symbiotic AI, under the NRRP MUR program supported by NextGenerationEU (CUP H97G22000210007), and the PHaSE project - Promoting Healthy and Sustainable Eating through Interactive and Explainable AI Methods, funded by MUR under the PRIN 2022 program - Finanziato dall'Unione Europea - NextGeneration EU, Missione 4 Componente 1 (CUP H53D23003530006).

CRediT authorship contribution statement

Matteo Vigone: Conceptualization, Methodology, Software, Writing – original draft, Writing – review & editing. **Alessandro Francesco Maria Martina:** Methodology, Writing – review & editing. **Cataldo Musto:** Supervision, Validation, Writing – review & editing. **Giovanni Semeraro:** Supervision, Validation.

A. Prompt General Structure

Table 10
Standardized CoT Prompting Structure applied across the framework.

#	Prompt Description
1	**Role Assumption:** Definition of the agent's persona and primary objective.
2	**Input Specification:** Structural definition of the incoming data parameters.
3	**Output Schema:** Strict formatting rules for the expected response.
4	**Domain Guidelines:** Context-specific heuristics (e.g., RESTfulness, SOLID principles).
5	**Technology Stack:** Preferred libraries, frameworks, and versions.
6	**Mandatory Constraints:** Actions or patterns that must be explicitly executed.
7	**Negative Constraints:** Exclusion criteria and patterns to avoid (e.g., "No placeholders").
8	**Failure Prevention:** Edge cases and critical warnings to mitigate known error modes.
9	**Execution Summary:** Final synopsis of the task logic.

B. Prompt for Evaluation

Prompt 1 (low specificity). I want you to generate a web app for my personal blog. I am a machine learning engineer. There are two main actors: me who is the owner and the visitors to my blog. I want the web app to have various sections: displaying data about who I am, the possibility to enter my projects and the possibility to enter articles I have published.

Prompt 2 (low-mid specificity). I am a machine learning engineer and want to create a web app for my personal blog. The web app must have two main roles: the administrator and the visitors. I want three main sections on the site: an “About Me” page with a detailed description of my background, a ‘Projects’ section where I upload my work with description, images and links, and an “Articles” section for my technical posts. The design should be similar to TailAdmin, using an elegant palette of white and light blue.

Prompt 3 (mid specificity). I’m a machine learning engineer and I want to build a personal blog web application where I can showcase my work and share technical insights. The app should have two primary user roles: myself as the admin (make login and signin), with the ability to manage content, and public visitors who can browse the content. The site should include three main sections: one page that introduces who I am, highlighting my experience and background in machine learning; a portfolio area to display my projects with detailed descriptions, tech stacks, screenshots, and external links; and a blog section for publishing technical articles. I’d like the UI to be inspired by TailAdmin, with a clean and modern design based on a color scheme that combines white and soft blue tones.

Prompt 4 (mid-high specificity). Create a personal web application for a machine learning engineer, designed to support two distinct user roles: the administrator (the owner of the blog) and the visitor. The application must be developed using React and TailwindCSS, with a layout inspired by the clean and structured design language of TailAdmin. The visual theme should rely on a light and professional aesthetic, using pure white as the primary background color, complemented by subtle shades of light blue for secondary surfaces (e0f2fe) and more vibrant blue tones (38bdf8) for active UI elements like buttons and hyperlinks. The homepage should feature an introductory section that includes a professional bio with a profile image, job title, and social media links (such as GitHub and LinkedIn). A dedicated "Projects" section should present each project through visually engaging cards that display the project name,

a short description, the tech stack used, and an external link to a live demo or repository. The "Articles" section should list blog posts, each with a title, abstract, publication date, and a link to the full content. The admin user must be able to access a restricted dashboard via a basic authentication system (login and registration). Inside the dashboard, the administrator can add and edit content—both projects and articles—through structured forms that appear as modals and support real-time validation. The entire user experience must be fully responsive, ensuring usability across desktop, tablet, and mobile devices, and offer smooth navigation for both the administrator and public visitors.

Prompt 5 (high specificity). I want you to generate a professional web app for my personal blog as a machine learning engineer. The web app must have two distinct roles: the administrator, which is me, with authenticated access and the ability to create, edit and delete content, and the visitors, who must be able to freely navigate the site in read-only mode. Make login and signin pages for authentication of admin. The user interface should be strongly inspired by TailAdmin's clean, modular, dashboard-like design, but should be customized with a palette based on pure white as the main background color (#ffffff), accompanied by light blue tones for visual accents (#bae6fd for secondary backgrounds, #0ea5e9 for buttons, links, and hovers). Typography must be modern, ideally using the "Inter" font in various weights. The layout should be fully responsive, optimized for desktop, tablet and mobile, with a vertical sidebar for persistent navigation in desktop views and a hamburger menu in mobile views. The home page must feature an introductory section with my profile photo, name, job title, a brief professional bio, and links to my LinkedIn and GitHub profiles. Next, there must be a section devoted to projects, shown through visually appealing cards that include the project name, a brief description, the tech stack used and a clickable link to the online demo or GitHub repository. Further down, I want a blog section where published articles are shown in a sortable list by date, with a preview of the content, title, subtitle, and a button to read the entire article on a dedicated page. I also want you to create a simple authentication system based on login and registration, allowing access to a restricted dashboard. This dashboard must allow me to add and edit new articles and projects and edit bio description through modal forms with real-time validation. Add the possibility of enabling an optional dark mode, maintaining color consistency with the defined palette.

References

Acharya, M., Zhang, Y., Leach, K., Huang, Y., 2025. Optimizing Code Runtime Performance Through Context-aware Retrieval-augmented Generation, in: 33rd IEEE/ACM International Conference on Program Comprehension, ICPC at ICSE 2025, Ottawa, ON, Canada, April 27-28, 2025, IEEE. pp. 1–5. URL: https://doi.org/10.1109/ICPC66645.2025.00028, doi:10.1109/ICPC66645.2025.00028.

Austin, J., Odena, A., Nye, M.I., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C.J., Terry, M., Le, Q.V., Sutton, C., 2021. Program Synthesis with Large Language Models. CoRR abs/2108.07732. URL: https://arxiv.org/abs/2108.07732, arXiv:2108.07732.

Bairi, R., Sonwane, A., Kanade, A., C., V.D., Iyer, A., Parthasarathy, S., Rajamani, S.K., Ashok, B., Shet, S., 2024. CodePlan: Repository-level Coding using LLMs and Planning. Proc. ACM Softw. Eng. 1, 675–698. URL: https://doi.org/10.1145/3643757, doi:10.1145/3643757.

Banerjee, S., Lavie, A., 2005. METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments, in: Goldstein, J., Lavie, A., Lin, C., Voss, C.R. (Eds.), Proceedings of the Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization at ACL 2005, Ann Arbor, Michigan, USA, June 29, 2005, Association for Computational Linguistics. pp. 65–72. URL: https://aclanthology.org/W05-0909/.

Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D., 2020. Language Models are Few-shot Learners, in: Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., Lin, H. (Eds.), Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual. URL: https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html.

Caldiera, V.R.B.G., Rombach, H.D., 1994. The goal question metric approach. Encyclopedia of software engineering , 528–532.

Chang, T., Chen, S., Fan, G., Feng, Z., 2023. A Self-iteration Code Generation Method Based on Large Language Models, in: 29th IEEE International Conference on Parallel and Distributed Systems, ICPADS 2023, Ocean Flower Island, China, December 17-21, 2023, IEEE. pp. 275–281. URL: https://doi.org/10.1109/ICPADS60453.2023.00049, doi:10.1109/ICPADS60453.2023.00049.

Chen, A., Phang, J., Parrish, A., Padmakumar, V., Zhao, C., Bowman, S.R., Cho, K., 2024a. Two Failures of Self-consistency in the Multi-step Reasoning of LLMs. Trans. Mach. Learn. Res. 2024. URL: https://openreview.net/forum?id=5nBqY1y96B.

Chen, L., Guo, Q., Jia, H., Zeng, Z., Wang, X., Xu, Y., Wu, J., Wang, Y., Gao, Q., Wang, J., Ye, W., Zhang, S., 2024b. A Survey on Evaluating Large Language Models in Code Generation Tasks. CoRR abs/2408.16498. URL: https://doi.org/10.48550/arXiv.2408.16498, doi:10.48550/ARXIV.2408.16498, arXiv:2408.16498.

Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H.P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F.P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W.H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A.N., Leike, J., Achiam, J., Misra, V., Morikawa, E.,

Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., Zaremba, W., 2021. Evaluating Large Language Models Trained on Code. CoRR abs/2107.03374. URL: https://arxiv.org/abs/2107.03374, arXiv:2107.03374.

Chen, X., Lin, M., Schärli, N., Zhou, D., 2024c. Teaching Large Language Models to Self-debug, in: The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024, OpenReview.net. URL: https://openreview.net/forum?id=KuPixIqPiq.

Dikert, K., Paasivaara, M., Lassenius, C., 2016. Challenges and success factors for large-scale agile transformations: A systematic literature review. J. Syst. Softw. 119, 87–108. URL: https://doi.org/10.1016/j.jss.2016.06.013, doi:10.1016/J.JSS.2016.06.013.

Dong, Q., Li, L., Dai, D., Zheng, C., Ma, J., Li, R., Xia, H., Xu, J., Wu, Z., Chang, B., Sun, X., Li, L., Sui, Z., 2024. A Survey on In-context Learning, in: Al-Onaizan, Y., Bansal, M., Chen, Y. (Eds.), Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024, Association for Computational Linguistics. pp. 1107–1128. URL: https://doi.org/10.18653/v1/2024.emnlp-main.64, doi:10.18653/V1/2024.EMNLP-MAIN.64.

Dong, Y., Jiang, X., Qian, J., Wang, T., Zhang, K., Jin, Z., Li, G., 2025. A Survey on Code Generation with LLM-based Agents. CoRR abs/2508.00083. URL: https://doi.org/10.48550/arXiv.2508.00083, doi:10.48550/ARXIV.2508.00083, arXiv:2508.00083.

Du, M., Luu, A.T., Ji, B., Liu, Q., Ng, S., 2024. Mercury: A Code Efficiency Benchmark for Code Large Language Models, in: Globersons, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J.M., Zhang, C. (Eds.), Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024. URL: http://papers.nips.cc/paper_files/paper/2024/hash/1df1df43b58845650b8dada00fca9772-Abstract-Datasets_and_Benchmarks_Track.html.

Evtikhiev, M., Bogomolov, E., Sokolov, Y., Bryksin, T., 2023. Out of the bleu: How should we assess quality of the code generation models? Journal of Systems and Software 203, 111741. URL: https://www.sciencedirect.com/science/article/pii/S016412122300136X, doi:https://doi.org/10.1016/j.jss.2023.111741.

Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., Zhou, M., 2020. CodeBERT: A Pre-trained Model for Programming and Natural Languages, in: Cohn, T., He, Y., Liu, Y. (Eds.), Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020, Association for Computational Linguistics. pp. 1536–1547. URL: https://doi.org/10.18653/v1/2020.findings-emnlp.139, doi:10.18653/V1/2020.FINDINGS-EMNLP.139.

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Guo, Q., Wang, M., Wang, H., 2023. Retrieval-augmented Generation for Large Language Models: A Survey. CoRR abs/2312.10997. URL: https://doi.org/10.48550/arXiv.2312.10997, doi:10.48550/ARXIV.2312.10997, arXiv:2312.10997.

Gonzalez, E.A., Rothkopf, R., Lerner, S., Polikarpova, N., 2025. HiLDe: Intentional Code Generation via Human-in-the-loop Decoding. CoRR abs/2505.22906. URL: https://doi.org/10.48550/arXiv.2505.22906, doi:10.48550/ARXIV.2505.22906, arXiv:2505.22906.

Guo, D., Xu, C., Duan, N., Yin, J., McAuley, J.J., 2023. LongCoder: A Long-range Pre-trained Language Model for Code Completion, in: Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., Scarlett, J. (Eds.), International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA, PMLR. pp. 12098–12107. URL: https://proceedings.mlr.press/v202/guo23j.html.

Gur, I., Furuta, H., Huang, A.V., Safdari, M., Matsuo, Y., Eck, D., Faust, A., 2024. A Real-world WebAgent with Planning, Long Context Understanding, and Program Synthesis, in: The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024, OpenReview.net. URL: https://openreview.net/forum?id=9JQtrumvg8.

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S.K.S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., Schmidhuber, J., 2024. MetaGPT: Meta Programming for A Multi-agent Collaborative Framework, in: The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024, OpenReview.net. URL: https://openreview.net/forum?id=VtmBAGCN7o.

Hu, Y., Zhou, Q., Chen, Q., Li, X., Liu, L., Zhang, D., Kachroo, A., Oz, T., Tripp, O., 2025. QualityFlow: An Agentic Workflow for Program Synthesis Controlled by LLM Quality Checks. CoRR abs/2501.17167. URL: https://doi.org/10.48550/arXiv.2501.17167, doi:10.48550/ARXIV.2501.17167, arXiv:2501.17167.

Huang, D., Bu, Q., Zhang, J.M., Luck, M., Cui, H., 2023. AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation. CoRR abs/2312.13010. URL: https://doi.org/10.48550/arXiv.2312.13010, doi:10.48550/ARXIV.2312.13010, arXiv:2312.13010.

Huang, D., Qing, Y., Shang, W., Cui, H., Zhang, J., 2024. EffiBench: Benchmarking the Efficiency of Automatically Generated Code, in: Globersons, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J.M., Zhang, C. (Eds.), Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024. URL: http://papers.nips.cc/paper_files/paper/2024/hash/15807b6e09d691fe5e96cdecde6d7b80-Abstract-Datasets_and_Benchmarks_Track.html.

Ishibashi, Y., Nishimura, Y., 2024. Self-organized Agents: A LLM Multi-agent Framework toward Ultra Large-scale Code Generation and Optimization. CoRR abs/2404.02183. URL: https://doi.org/10.48550/arXiv.2404.02183, doi:10.48550/ARXIV.2404.02183, arXiv:2404.02183.

Islam, M.A., Ali, M.E., Parvez, M.R., 2024. MapCoder: Multi-agent Code Generation for Competitive Problem Solving, in: Ku, L., Martins, A., Srikumar, V. (Eds.), Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024, Association for Computational Linguistics. pp. 4912–4944. URL: https://doi.org/10.18653/v1/2024.acl-long.269, doi:10.18653/V1/2024.ACL-LONG.269.

Jain, R., Suman, U., 2015. A Systematic Literature Review on Global Software Development Life Cycle. ACM SIGSOFT Softw. Eng. Notes 40, 1–14. URL: https://doi.org/10.1145/2735399.2735408, doi:10.1145/2735399.2735408.

Jiang, X., Dong, Y., Wang, L., Fang, Z., Shang, Q., Li, G., Jin, Z., Jiao, W., 2024. Self-planning Code Generation with Large Language Models. ACM Trans. Softw. Eng. Methodol. 33, 182:1–182:30. URL: https://doi.org/10.1145/3672456, doi:10.1145/3672456.

Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K.R., 2024. SWE-bench: Can language models resolve real-world github issues?, in: The Twelfth International Conference on Learning Representations. URL: https://openreview.net/forum?id=VTF8yNQM66.

Jitnah, D., Han, J., Steele, P., 1995. Software requirements engineering: An overview. Penins. Sch. Comput. Inf. Technol. Monash Univ 1995.

Kalai, A.T., Nachum, O., Vempala, S.S., Zhang, E., 2025. Why Language Models Hallucinate. CoRR abs/2509.04664. URL: https://doi.org/10.48550/arXiv.2509.04664, doi:10.48550/ARXIV.2509.04664, arXiv:2509.04664.

Li, B., Wu, W., Tang, Z., Shi, L., Yang, J., Li, J., Yao, S., Qian, C., Hui, B., Zhang, Q., et al., 2024a. Devbench: A comprehensive benchmark for software development. arXiv preprint arXiv:2403.08604 3.

Li, C., Liang, J., Zeng, A., Chen, X., Hausman, K., Sadigh, D., Levine, S., Fei-Fei, L., Xia, F., Ichter, B., 2024b. Chain of Code: Reasoning with a Language Model-augmented Code Emulator, in: Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024, OpenReview.net. URL: https://openreview.net/forum?id=vKtomqlSxm.

Li, J., Li, G., Li, Y., Jin, Z., 2025. Structured Chain-of-thought Prompting for Code Generation. ACM Trans. Softw. Eng. Methodol. 34, 37:1–37:23. URL: https://doi.org/10.1145/3690635, doi:10.1145/3690635.

Lin, C.Y., 2004. Rouge: A package for automatic evaluation of summaries, in: Text summarization branches out, pp. 74–81.

Lu, S., Duan, N., Han, H., Guo, D., Hwang, S.w., Svyatkovskiy, A., 2022. ReACC: A Retrieval-augmented Code Completion Framework, in: Muresan, S., Nakov, P., Villavicencio, A. (Eds.), Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Dublin, Ireland. pp. 6227–6240. URL: https://aclanthology.org/2022.acl-long.431/, doi:10.18653/v1/2022.acl-long.431.

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B.P., Hermann, K., Welleck, S., Yazdanbakhsh, A., Clark, P., 2023. Self-Refine: Iterative Refinement with Self-feedback, in: Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (Eds.), Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023. URL: http://papers.nips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html.

Mosbach, M., Pimentel, T., Ravfogel, S., Klakow, D., Elazar, Y., 2023. Few-shot Fine-tuning vs. In-context Learning: A Fair Comparison and Evaluation, in: Rogers, A., Boyd-Graber, J.L., Okazaki, N. (Eds.), Findings of the Association for Computational Linguistics: ACL 2023, Toronto, Canada, July 9-14, 2023, Association for Computational Linguistics. pp. 12284–12314. URL: https://doi.org/10.18653/v1/2023.findings-acl.779, doi:10.18653/V1/2023.FINDINGS-ACL.779.

Nunez, A., Islam, N.T., Jha, S.K., Najafirad, P., 2024. AutoSafeCoder: A Multi-agent Framework for Securing LLM Code Generation through Static Analysis and Fuzz Testing. CoRR abs/2409.10737. URL: https://doi.org/10.48550/arXiv.2409.10737, doi:10.48550/ARXIV.2409.10737, arXiv:2409.10737.

Odeh, A., Odeh, N., Mohammed, A.S., 2024. A comparative review of AI techniques for automated code generation in software development: advancements, challenges, and future directions. TEM Journal 13, 726.

Olausson, T.X., Inala, J.P., Wang, C., Gao, J., Solar-Lezama, A., 2024. Is Self-repair a Silver Bullet for Code Generation?, in: The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024, OpenReview.net. URL: https://openreview.net/forum?id=y0GJXRungR.

Pan, R., Zhang, H., Liu, C., 2025. CodeCoR: An LLM-based Self-reflective Multi-agent Framework for Code Generation. CoRR abs/2501.07811. URL: https://doi.org/10.48550/arXiv.2501.07811, doi:10.48550/ARXIV.2501.07811, arXiv:2501.07811.

Papineni, K., Roukos, S., Ward, T., Zhu, W., 2002. Bleu: a Method for Automatic Evaluation of Machine Translation, in: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, July 6-12, 2002, Philadelphia, PA, USA, ACL. pp. 311–318. URL: https://aclanthology.org/P02-1040/, doi:10.3115/1073083.1073135.

Pezeshkpour, P., Kandogan, E., Bhutani, N., Rahman, S., Mitchell, T., Hruschka, E., 2024. Reasoning capacity in multi-agent systems: Limitations, challenges and human-centered solutions. arXiv preprint arXiv:2402.01108 .

Phan, H.N., Phan, H.N., Nguyen, T.N., Bui, N.D.Q., 2024. RepoHyper: Better Context Retrieval Is All You Need for Repository-level Code Completion. CoRR abs/2403.06095. URL: https://doi.org/10.48550/arXiv.2403.06095, doi:10.48550/ARXIV.2403.06095, arXiv:2403.06095.

Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., Sun, M., 2024. ChatDev: Communicative Agents for Software Development, in: Ku, L., Martins, A., Srikumar, V. (Eds.), Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024, Association for Computational Linguistics. pp. 15174–15186. URL: https://doi.org/10.18653/v1/2024.acl-long.810, doi:10.18653/V1/2024.ACL-LONG.810.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al., 2019. Language models are unsupervised multitask learners. OpenAI blog 1, 9.

Rasheed, Z., Sami, M.A., Kemell, K.K., Waseem, M., Saari, M., Systä, K., Abrahamsson, P., 2024. Codepori: Large-scale system for autonomous software development using multi-agent technology. arXiv preprint arXiv:2402.01411 .

Reddy, E.J., Sridhar, C., Rangadu, V.P., 2015. Knowledge based engineering: notion, approaches and future trends. American Journal of Intelligent Systems 5, 1–17.

Ren, S., Guo, D., Lu, S., Zhou, L., Liu, S., Tang, D., Sundaresan, N., Zhou, M., Blanco, A., Ma, S., 2020. CodeBLEU: a Method for Automatic Evaluation of Code Synthesis. CoRR abs/2009.10297. URL: https://arxiv.org/abs/2009.10297, arXiv:2009.10297.

Shin, J., Nam, J., et al., 2021. A survey of automatic code generation from natural language. Journal of Information Processing Systems 17, 537–555.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S., 2023. Reflexion: Language agents with verbal reinforcement learning. Advances in neural information processing systems 36, 8634–8652.

Svyatkovskiy, A., Deng, S.K., Fu, S., Sundaresan, N., 2020. IntelliCode compose: code generation using transformer, in: Devanbu, P., Cohen, M.B., Zimmermann, T. (Eds.), ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020, ACM. pp. 1433–1443. URL: https://doi.org/10.1145/3368089.

3417058, doi:10.1145/3368089.3417058.

Terragni, V., Vella, A., Roop, P.S., Blincoe, K., 2025. The Future of AI-driven Software Engineering. ACM Trans. Softw. Eng. Methodol. 34, 120:1–120:20. URL: https://doi.org/10.1145/3715003, doi:10.1145/3715003.

Tonmoy, S., Zaman, S., Jain, V., Rani, A., Rawte, V., Chadha, A., Das, A., 2024. A comprehensive survey of hallucination mitigation techniques in large language models. arXiv preprint arXiv:2401.01313 6.

Ulan, M., Löwe, W., Ericsson, M., Wingkvist, A., 2021. Weighted software metrics aggregation and its application to defect prediction. Empirical Software Engineering 26, 86.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I., 2017. Attention is All you Need, in: Guyon, I., von Luxburg, U., Bengio, S., Wallach, H.M., Fergus, R., Vishwanathan, S.V.N., Garnett, R. (Eds.), Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA, pp. 5998–6008. URL: https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

Vijayvargiya, S., Zhou, X., Yerukola, A., Sap, M., Neubig, G., 2025. Interactive Agents to Overcome Ambiguity in Software Engineering. CoRR abs/2502.13069. URL: https://doi.org/10.48550/arXiv.2502.13069, doi:10.48550/ARXIV.2502.13069, arXiv:2502.13069.

Wang, X., Zhu, W., Saxon, M., Steyvers, M., Wang, W.Y., 2023. Large Language Models Are Latent Variable Models: Explaining and Finding Good Demonstrations for In-context Learning, in: Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (Eds.), Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023. URL: http://papers.nips.cc/paper_files/paper/2023/hash/3255a7554605a88800f4e120b3a929e1-Abstract-Conference.html.

Wang, Y., Wang, W., Joty, S.R., Hoi, S.C.H., 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-decoder Models for Code Understanding and Generation, in: Moens, M., Huang, X., Specia, L., Yih, S.W. (Eds.), Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021, Association for Computational Linguistics. pp. 8696–8708. URL: https://doi.org/10.18653/v1/2021.emnlp-main.685, doi:10.18653/V1/2021.EMNLP-MAIN.685.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E.H., Le, Q.V., Zhou, D., 2022. Chain-of-thought Prompting Elicits Reasoning in Large Language Models, in: Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., Oh, A. (Eds.), Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022. URL: http://papers.nips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html.

Yang, C., Liu, Y., Yin, C., 2021. Recent Advances in Intelligent Source Code Generation: A Survey on Natural Language Based Studies. Entropy 23, 1174. URL: https://doi.org/10.3390/e23091174, doi:10.3390/E23091174.

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., Narasimhan, K., 2023. Tree of thoughts: Deliberate problem solving with large language models. Advances in neural information processing systems 36, 11809–11822.

Zhang, H., Cheng, W., Wu, Y., Hu, W., 2024. A Pair Programming Framework for Code Generation via Multi-plan Exploration and Feedback-driven Refinement, in: Filkov, V., Ray, B., Zhou, M. (Eds.), Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024, Sacramento, CA, USA, October 27 - November 1, 2024, ACM. pp. 1319–1331. URL: https://doi.org/10.1145/3691620.3695506, doi:10.1145/3691620.3695506.

Zhang, K., Li, Z., Li, J., Li, G., Jin, Z., 2023. Self-Edit: Fault-aware Code Editor for Code Generation, in: Rogers, A., Boyd-Graber, J.L., Okazaki, N. (Eds.), Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9-14, 2023, Association for Computational Linguistics. pp. 769–787. URL: https://doi.org/10.18653/v1/2023.acl-long.45, doi:10.18653/V1/2023.ACL-LONG.45.

Zheng, L., Chiang, W., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E.P., Zhang, H., Gonzalez, J.E., Stoica, I., 2023. Judging LLM-as-a-judge with MT-bench and Chatbot Arena, in: Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (Eds.), Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023. URL: http://papers.nips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html.